

MealMate

Online Food Ordering & Delivery System

A Django-based web application with Razorpay payment integration

Project Name	MealMate
Type	Web Application (Django)
Domain	Online Food Ordering & Delivery
Database	SQLite
Payment Gateway	Razorpay
Document	Project Presentation Report

Table of Contents

1	Introduction
2	Problem Statement
3	Objectives
4	Technology Stack
5	System Architecture
6	Modules
7	Database Models
8	Key Features
9	URL Routing Highlights
10	Workflow - Customer
11	Workflow - Admin
12	Advantages
13	Limitations
14	Future Enhancements
15	Conclusion

1. Introduction

- MealMate is an online food ordering and delivery web application.
- Built using the Django web framework (Python) with SQLite as the database.
- Supports two roles: Admin (manages restaurants & menus) and Customer (browses, orders, pays).
- Integrated with the Razorpay payment gateway for secure online payments.

2. Problem Statement

- Customers need a simple, centralized platform to browse multiple restaurants and order food.
- Restaurants need a manageable way to maintain their menus and prices online.
- Manual ordering processes (phone / in-person) are slow and error-prone.
- Need for a secure online payment workflow with order confirmation.

3. Objectives

- Provide user authentication (sign up / sign in) for customers and admins.
- Enable admins to add, update, and delete restaurants and menu items.
- Allow customers to browse restaurants, view menus, and add items to a cart.
- Facilitate secure checkout via Razorpay and display an order confirmation.

4. Technology Stack

- Backend: Python 3, Django 5.1.14
- Frontend: HTML5, CSS3, Django Template Language
- Database: SQLite (default Django DB)
- Payment Gateway: Razorpay (razorpay 2.0.1 SDK)
- Other Tools: Django Admin, Django ORM, Django URL routing

5. System Architecture

- Client (Browser) <-> Django Views <-> Django ORM <-> SQLite Database
- Templates render dynamic HTML pages using context data.

- Razorpay client created in checkout view to generate an order ID.
- Frontend payment widget collects payment, then redirects to orders page.
- MVC-style separation: Models (data), Views (logic), Templates (presentation).

6. Modules

- Authentication: signup, signin, role-based redirection (admin vs. customer).
- Restaurant Management (Admin): add / update / delete restaurants.
- Menu Management (Admin): add menu items linked to a restaurant.
- Customer Browsing: view restaurant list and per-restaurant menu.
- Cart: add items to cart, view cart, compute total price.
- Checkout & Payment: Razorpay order creation & confirmation page.

7. Database Models

- Customer: username, password, email, mobile, address.
- Restaurant: name, picture (URL), cuisine, rating.
- Item: restaurant (FK), name, description, price, vegetarian flag, picture.
- Cart: customer (FK), items (M2M to Item), total_price() helper.
- Relationships: Restaurant 1<->N Item ; Customer 1<->1 Cart ; Cart N<->N Item.

8. Key Features

- Two-tier login flow: 'admin' user routes to admin dashboard, others to customer home.
- Restaurant catalog with images, cuisine type, and rating.
- Per-restaurant menu with vegetarian / non-vegetarian flag.
- Add-to-cart, cart total computation, and clear-cart-on-order behavior.
- Razorpay-powered online payments in INR (test mode keys configured).

9. URL Routing Highlights

- / Landing page
- /open_signin, /signin Authentication
- /open_signup, /signup Customer registration

- /open_add_restaurant Admin: add restaurant page
- /view_menu// Customer: view a restaurant's menu
- /show_cart/ Customer: cart page
- /checkout// Razorpay checkout
- /orders// Order confirmation

10. Workflow - Customer

- 1. Sign up / Sign in.
- 2. View list of restaurants on the home page.
- 3. Open a restaurant's menu and add items to cart.
- 4. Open cart, review items and total price.
- 5. Proceed to checkout (Razorpay payment).
- 6. On success, view orders confirmation; cart is cleared.

11. Workflow - Admin

- 1. Sign in as 'admin'.
- 2. Add a new restaurant (name, picture URL, cuisine, rating).
- 3. View all restaurants and update or delete any of them.
- 4. Update menu of a restaurant by adding new items.
- 5. Items are linked to their parent restaurant via a foreign key.

12. Advantages

- Centralized platform for multiple restaurants and customers.
- Quick and convenient food ordering experience.
- Secure online payment via a trusted gateway (Razorpay).
- Simple admin interface to keep restaurant data up to date.
- Easy to extend (new fields, new modules) thanks to Django's ORM.

13. Limitations

- Passwords are stored as plain text (CharField) - not production safe.

- No real authentication / session management built on Django auth.
- No order history persistence (cart is cleared on confirmation).
- No delivery tracking, ratings, or reviews from customers yet.
- Static media only via image URLs - no file uploads.

14. Future Enhancements

- Use Django's built-in auth system with hashed passwords.
- Persist orders in a dedicated Order / OrderItem model with status tracking.
- Add delivery agent module and live order tracking.
- Allow customers to rate and review restaurants and items.
- Mobile-responsive UI and a REST API for a future mobile app.
- Deploy to a cloud platform (PythonAnywhere / Render / AWS).

15. Conclusion

- MealMate demonstrates a complete food-ordering workflow on Django.
- Covers user roles, CRUD operations, cart management, and online payments.
- Provides a solid foundation for scaling into a production-grade system.
- Showcases practical use of Django ORM, templates, and third-party integrations.

Thank You

Questions & Discussion